Measuring What Quantization Changes: A Reproducible Latency, Parity and Quality Benchmark of PyTorch and ONNX Runtime for Edge Vision Models

Utkarsh Sengar
Department of Computer Science and Engineering
Indian Institute of Technology Dharwad
Dharwad, Karnataka, India
cs23bt026@iitdh.ac.in

***Abstract*—Choosing a runtime and numeric precision for an edge vision model is usually justified by latency alone. We present an open, fully regenerable benchmark that measures, for every (model, runtime, precision) configuration, pooled latency percentiles with bootstrap confidence intervals and a between-session stability verdict, a measured end-to-end stage breakdown, the numerical parity of all outputs against a PyTorch FP32 reference, and task quality re-evaluated through the runtime itself, against acceptance gates fixed before the run. Evaluating a compact anchor-free detector and a convolutional autoencoder, both trained from scratch, in PyTorch (CPU, GPU) and ONNX Runtime (CPU) at FP32, FP16 and static INT8, we find that INT8 is model-dependent ($1.53\times$ faster and 73.6% smaller for the detector, but only $1.04\times$ for the autoencoder), that default TF32 GPU math breaks FP32 parity, and that host-side pre- and post-processing dominate the detector's end-to-end latency (10.21 ms versus 1.61 ms model path). We also test quantization-aware NMS calibration: re-tuning post-processing thresholds raises detection F_1 from 0.089 to 0.712 for every precision, but INT8-specific thresholds give no significant gain over reusing the FP32-calibrated ones ($\Delta F_1 = -0.008$, 95% CI $[-0.029, +0.013]$), while decision-flip attribution shows that INT8 changes 0.273 of individual detection decisions.**

***Index Terms*—Edge inference, ONNX Runtime, post-training quantization, benchmarking methodology, object detection, anomaly detection**

I. Introduction

Deploying a vision model means choosing a runtime and a numeric precision, and the choice is usually justified by latency numbers alone. Two things are easy to get wrong in that process. First, latency on a general-purpose host is noisy: medians move between measurement sessions, and a single session can make an optimization look better or worse than it is. Second, lower precision changes *decisions*, not only tensors: for a detector, a small logit drift can push boxes across the confidence threshold or past non-maximum suppression (NMS), which an aggregate metric such as mAP may hide.

This paper presents an open benchmark that treats latency, numerical parity and task quality as one measurement. For every (model, runtime, precision) configuration it reports pooled latency percentiles with bootstrap confidence intervals and a between-session stability verdict, a measured end-to-end stage breakdown, the parity of every output tensor against a PyTorch FP32 reference, and the task metric re-evaluated through that exact runtime, each checked against acceptance thresholds fixed in configuration files before the run. On top of this we evaluate *Q-Aware NMS*—re-selecting the post-processing thresholds per precision on a labeled calibration split—and *decision-flip attribution*, which decomposes how reduced precision changes individual detections.

Contributions: (i) a session-based protocol and open implementation in which every reported number is regenerated by one command; (ii) an evaluation of FP32, FP16 and static INT8 inference in PyTorch and ONNX Runtime [1] for a compact anchor-free detector and a convolutional autoencoder; (iii) a statistically tested assessment of per-precision threshold calibration; and (iv) a detection-level flip analysis of quantization.

All models are trained from scratch on procedurally generated data, and all measurements come from a single laptop workstation. The results therefore characterize the runtimes and the methodology, not real-world accuracy or embedded hardware; Section V discusses these limits.

II. Related Work

Quantization. Integer quantization maps tensors to 8-bit values via a scale and zero point [2]. Post-training quantization (PTQ) calibrates the ranges on a small dataset without retraining and is the common deployment path [3]–[5]; surveys of compression and acceleration discuss the resulting accuracy–efficiency trade-offs [6]. **Dense detection.** One-stage detectors predict boxes on a dense grid [7], [8] and rely on NMS [9], [10]; focal and quality-aware classification losses [11], [12] and IoU-based regression losses [13] are standard. Evaluation follows the COCO protocol [14]. **Anomaly detection.** Reconstruction-based models score defects by reconstruction error and are evaluated with image AUROC and the per-region-overlap (PRO) curve [15]. **Measurement.** Robust statistics such as the median absolute deviation [16], bootstrap

confidence intervals [17] and rank tests with family-wise error control [18], [19] are standard tools that inference benchmarks rarely report.

III. Benchmark Design

A. Workloads and data

The **detector** is a compact anchor-free network (1.95 M parameters): a five-stage strided Conv-BN-SiLU backbone and three heads at strides 8/16/32 that predict per-location (left, top, right, bottom) distances [8], decoded inside the exported graph to an output of shape $[1, 4{+}2, 8400]$ for a $640{\times}640$ input. It is trained for 12000 steps with a quality focal loss [12] and a GIoU loss [13]. The **autoencoder** (1.42 M parameters) has a four-stage strided-convolution encoder, a $1{\times}1$ bottleneck and a transposed-convolution decoder; it is trained for 2500 steps on defect-free textures only, and its anomaly map is the per-pixel mean absolute reconstruction error.

Data are procedural: two-class shapes (rectangles and ellipses) with unlabeled distractors on noisy textured backgrounds of four resolutions, and sinusoidal surface textures with scratch, stain and texture-dropout defects with pixel masks. Train, validation, calibration and test splits come from disjoint seed ranges, and calibration/test file overlap is verified to be zero. The validation split is used only to select the anomaly-score smoothing ($\sigma{=}2$ px Gaussian, mean of the top 0.1% pixels). The test split contains 200 detection images (684 boxes) and 100 normal plus 100 anomalous textures.

B. Artifacts and runtimes

Models are exported to ONNX (opset 17) and simplified; FP16 graphs are converted with onnxconverter-common (FP32 graph inputs/outputs for the detector, FP16 for the autoencoder) and PyTorch FP16 uses `half()`; INT8 graphs use ONNX Runtime static QDQ quantization with MinMax calibration on the calibration split, per-channel symmetric weights, and Conv, ConvTranspose, MatMul and Gemm quantized. The matrix covers PyTorch eager on CPU and CUDA (FP32, FP16) and ONNX Runtime on CPU (FP32, FP16, INT8). ONNX Runtime CUDA and TensorRT back ends are implemented and are run automatically when available; neither was available on the benchmark host.

C. Measurement protocol

Model-path latency is measured on device-resident inputs: per session, 50 untimed and 300 timed passes with garbage collection disabled, timed with CUDA events on GPU (the call does not synchronize the host) and `perf_counter_ns` on CPU. Each configuration runs 5 sessions separated by a cache-eviction write and a cooldown; if the coefficient of variation (CV) of the session medians exceeds 0.04, two supplemental sessions are added once. Percentiles are computed over the pooled iterations with percentile-bootstrap 95% CIs ($B{=}2000$), and a configuration is *stable* if the CV of all session medians is at most 0.05. **End-to-end latency** times decoding, preprocessing, inference through the host-array API (including host–device copies) and post-processing separately over 200 iterations. **Parity** compares all output tensors with PyTorch FP32 on CPU for a real test image (maximum relative error and cosine similarity), and **quality** re-evaluates the full test split through the runtime under test. Acceptance thresholds per precision tier are fixed in configuration files (relative error $10^{-4}/5{\cdot}10^{-3}/0.1$; quality drop $0.001/0.005/0.02$ for FP32/FP16/INT8).

D. Q-Aware NMS and decision flips

For each detector configuration, raw outputs on the calibration split (100 images) are cached and $(\tau_c^*, \tau_i^*) = \arg\max F_1$ is searched over $\tau_c \in \{0.10, \ldots, 0.95\}$ and $\tau_i \in \{0.05, \ldots, 0.70\}$ in steps of 0.05 (252 points); ties keep the default $(0.25, 0.45)$. On the test split we report micro-F_1 at both operating points, a paired image-level bootstrap 95% CI of ΔF_1, and a two-sided Wilcoxon signed-rank test on per-image error counts (false positives + false negatives), Holm–Bonferroni corrected across configurations.

Decision flips compare a configuration's detections with those of the PyTorch FP32 reference. Boxes are matched greedily in score order (IoU ≥ 0.5, same class); unmatched reference boxes are *lost true positives* or *suppressed false positives* and unmatched target boxes are *new true positives* or *new false positives*, judged against ground truth. With M matched boxes and F flips, the flip rate is $\Phi = F/(M + F)$.

IV. Results

All numbers come from one benchmark session (`session_20261027T011458Z`) on a laptop with an NVIDIA GeForce RTX 4050 Laptop GPU (driver 581.86, CUDA 12.1) and an 13th Gen Intel(R) Core(TM) i5-13450HX (8 cores / 16 threads visible to the OS), running Ubuntu 24.04.4 LTS, Linux 6.6.87.2-microsoft-standard-WSL2 (x86_64) with Python 3.13.9, PyTorch 2.5.1+cu121 and ONNX Runtime 1.23.2. Of 12 configurations, 11 met the stability criterion and 11 passed both acceptance gates (Table I).

A. Models and parity

The trained detector reaches mAP@50 0.701 (mAP@50–95 0.501) on the test split and the autoencoder an image AUROC of 0.931 (AU-PRO 0.801). ONNX export is exact up to floating-point reordering (maximum relative error over all FP32 rows $1.6 \cdot 10^{-6}$) and FP16 stays within $3.7 \cdot 10^{-3}$ for both models. With PyTorch's default TF32 convolutions the GPU detector deviates by $1.7 \cdot 10^{-4}$, failing the FP32 gate (10^{-4}); the FP32 rows therefore use strict IEEE FP32 (TF32 disabled), which costs speed: the autoencoder takes 3.90 ms in FP32 versus 1.01 ms in FP16.

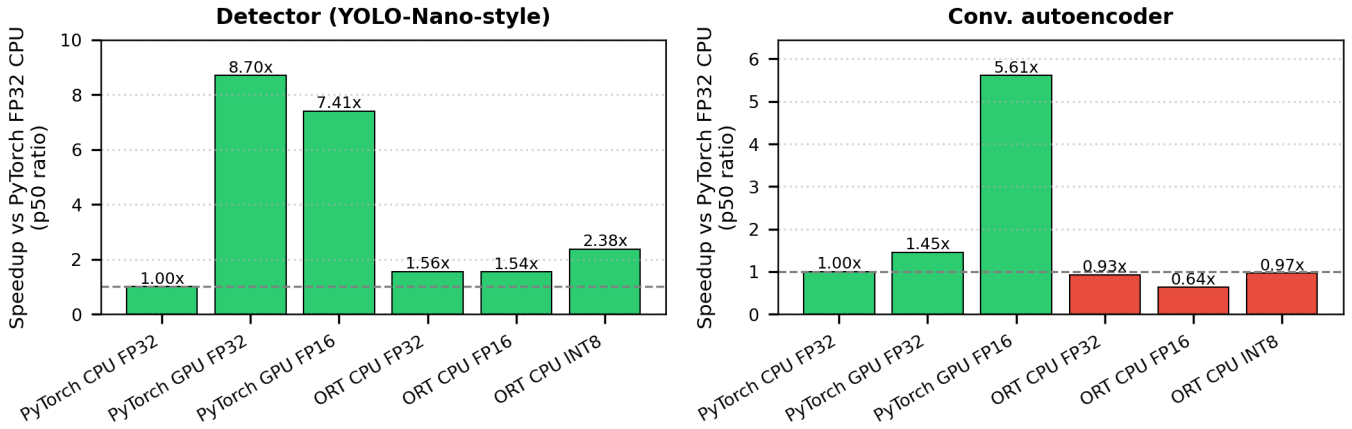
B. Latency

On the CPU, ONNX Runtime FP32 is $1.56\times$ faster than PyTorch for the detector (8.98 vs 13.98 ms) but $0.93\times$ for the autoencoder (6.13 vs 5.68 ms), i.e. the runtime choice is model-dependent (Fig. 1). FP16 graphs bring no CPU speedup ($0.99\times$ and $0.69\times$ vs ORT FP32) because the CPU lacks

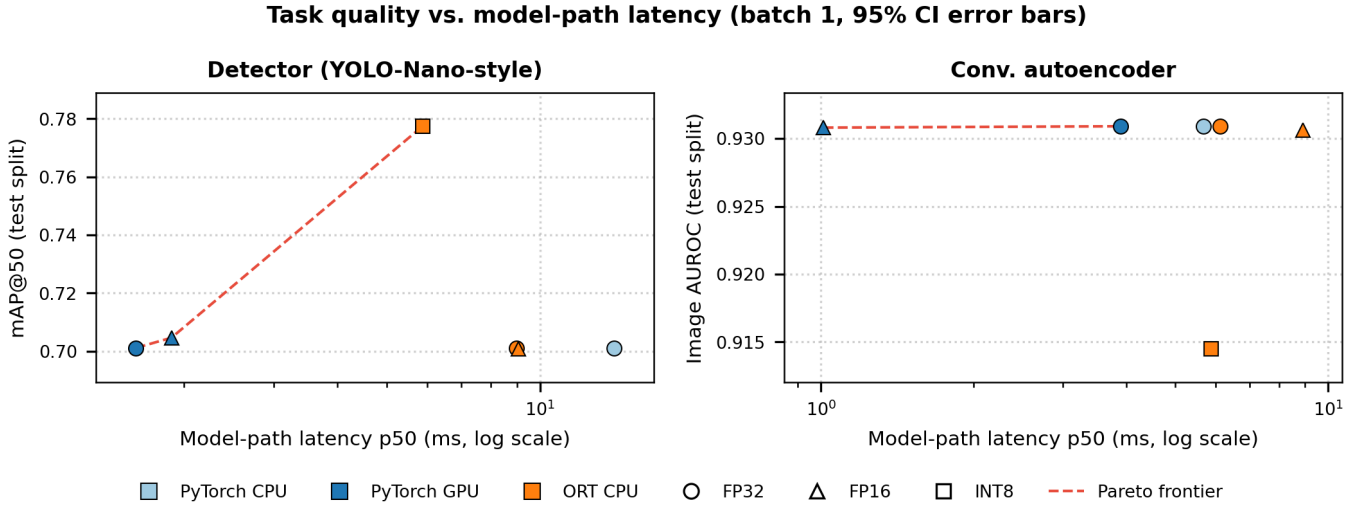
TABLE I

BATCH-1 RESULTS PER CONFIGURATION: POOLED MODEL-PATH LATENCY (MS) WITH BOOTSTRAP 95% CI, END-TO-END P50, SESSION-MEDIAN CV, MAXIMUM RELATIVE OUTPUT ERROR VS PYTORCH FP32 (CPU), TASK QUALITY ON THE TEST SPLIT (DETECTOR: MAP@50; AUTOENCODER: IMAGE AUROC) WITH THE DELTA TO THE FP32 REFERENCE, AND MODEL SIZE (MB = 2^{20} BYTES).

Model	Runtime	Prec.	p50 [95% CI] (ms)	p95	E2E p50	CV	Rel. err.	Quality (Δ)	MB
Detector	PyTorch CPU	FP32	13.98 [13.95, 14.05]	18.09	20.60	0.011	0.0e+00	0.701 (+0.000)	7.43
Detector	PyTorch GPU	FP32	1.61 [1.60, 1.62]	3.59	10.21	0.051	4.2e-07	0.701 (+0.000)	7.43
Detector	PyTorch GPU	FP16	1.89 [1.88, 1.89]	4.57	10.15	0.013	2.1e-03	0.705 (+0.004)	3.72
Detector	ORT CPU	FP32	8.98 [8.94, 9.01]	11.85	17.67	0.016	5.5e-07	0.701 (+0.000)	7.50
Detector	ORT CPU	FP16	9.08 [9.05, 9.11]	12.33	18.05	0.015	5.9e-04	0.701 (+0.000)	3.76
Detector	ORT CPU	INT8	5.88 [5.85, 5.91]	7.00	14.62	0.008	2.0e-01	0.777 (+0.076)	1.98
Autoencoder	PyTorch CPU	FP32	5.68 [5.65, 5.71]	7.29	8.09	0.023	0.0e+00	0.931 (+0.000)	5.41
Autoencoder	PyTorch GPU	FP32	3.90 [3.90, 3.90]	4.14	6.50	0.002	1.6e-06	0.931 (+0.000)	5.41
Autoencoder	PyTorch GPU	FP16	1.01 [1.01, 1.02]	1.45	3.39	0.020	3.7e-03	0.931 (+0.000)	2.70
Autoencoder	ORT CPU	FP32	6.13 [6.11, 6.16]	7.83	7.90	0.008	1.2e-06	0.931 (+0.000)	5.40
Autoencoder	ORT CPU	FP16	8.92 [8.90, 8.95]	10.46	11.18	0.006	1.9e-03	0.931 (+0.000)	2.70
Autoencoder	ORT CPU	INT8	5.88 [5.83, 5.92]	7.37	7.66	0.029	8.3e-02	0.914 (-0.016)	1.39



(a) Speedup vs PyTorch FP32 on CPU



(b) Quality vs model-path latency

Fig. 1. Model-path speedups and task quality versus latency (batch 1; 95% CIs as error bars).

native FP16 arithmetic. On the GPU the detector is launch-bound: FP16 (1.89 ms) is not faster than FP32 (1.61 ms), whereas the autoencoder gains 3.9×. The only configuration that missed the stability criterion was the detector on PyTorch

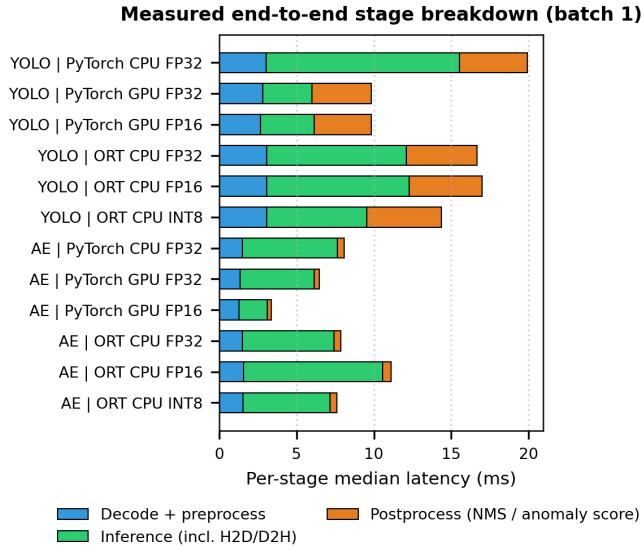


Fig. 2. Measured end-to-end stage medians (decode + preprocess, inference including host–device copies, post-processing).

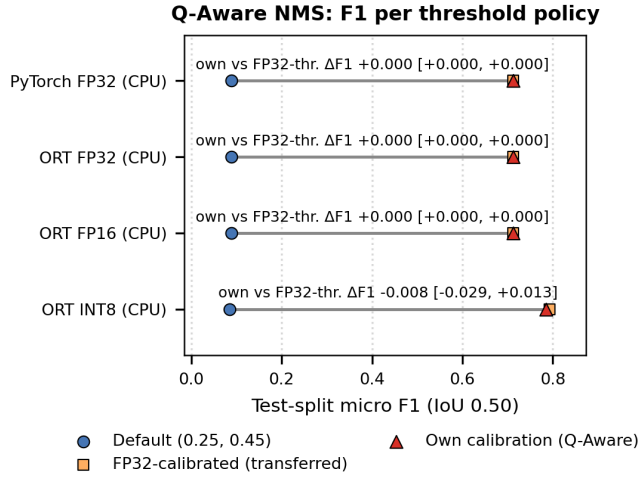


Fig. 3. Detection F_1 at default, FP32-calibrated and per-configuration calibrated post-processing thresholds.

GPU FP32 (session CV 0.051).

End to end, model execution is a minority of the detector's latency on the GPU (Fig. 2): decode and preprocessing take 2.81 ms, inference including copies 3.20 ms and NMS 3.81 ms, for an end-to-end p50 of 10.21 ms. NMS is expensive here because the detector emits many low-confidence candidates at the default threshold (precision 0.046 at recall 0.962), which also motivates the threshold study below.

C. Static INT8

The INT8 calibration method was chosen on the calibration split among MinMax, Percentile and Entropy; Percentile won for both models. With symmetric activation quantization, ONNX Runtime's Entropy ranges reduce to the MinMax scales (the resulting graphs are byte-identical), because the scale is set by the larger of the two asymmetric thresholds. For the detector, INT8 is $1.53\times$ faster than ORT FP32 and the file is 73.6% smaller (1.98 vs 7.50 MB), but its outputs deviate strongly (maximum relative error 2.0e-01), failing the parity gate. Its test mAP@50 nevertheless rose to 0.777 (+0.076): clipping changed the ranking of the many low-confidence boxes. We report this as a deviation, not an improvement; it would not necessarily transfer to other data. For the autoencoder INT8 is 74.3% smaller but only $1.04\times$ faster, and changes image AUROC by -0.016, within the 0.02 allowance.

TABLE II
TEST-SPLIT MICRO-F_1 AT THE DEFAULT THRESHOLDS (0.25, 0.45), WITH THE THRESHOLDS CALIBRATED ON THE FP32 REFERENCE (FP32-THR.), AND WITH EACH CONFIGURATION'S OWN CALIBRATION; THE PRECISION-SPECIFIC EFFECT IS OWN VS FP32-THR. (PAIRED BOOTSTRAP CI; WILCOXON ON PER-IMAGE ERRORS, HOLM-CORRECTED).

Configuration	(τ_c^*, τ_i^*)	F1 def.	F1 FP32-thr.	F1 own	Δ own vs FP32-thr. [95% CI]	p_{Holm}
PyTorch FP32 (CPU)	(0.80, 0.05)	0.089	0.712	0.712	+0.000 [+0.000, +0.000]	1
ORT FP32 (CPU)	(0.80, 0.05)	0.089	0.712	0.712	+0.000 [+0.000, +0.000]	1
ORT FP16 (CPU)	(0.80, 0.05)	0.088	0.712	0.712	+0.000 [+0.000, +0.000]	1
ORT INT8 (CPU)	(0.75, 0.10)	0.085	0.794	0.785	-0.008 [-0.029, +0.013]	0.35

TABLE III
DETECTION-LEVEL FLIPS AGAINST PYTORCH FP32 (TEST SPLIT).

Target	Thr.	Matched	Φ	Lost TP	New TP	New FP	Supp. FP
ORT FP32 (CPU)	default	14185	0.000	0	0	0	0
ORT FP32 (CPU)	calibrated	731	0.000	0	0	0	0
ORT FP16 (CPU)	default	14143	0.006	0	1	49	42
ORT FP16 (CPU)	calibrated	731	0.000	0	0	0	0
ORT INT8 (CPU)	default	9854	0.490	18	10	5134	4313
ORT INT8 (CPU)	calibrated	604	0.273	11	38	62	116

D. Q-Aware NMS

Calibrating the two post-processing thresholds on the calibration split is decisive for this detector: F_1 rises from 0.089 to 0.712 for the FP32 reference (Table II, Fig. 3; Holm-corrected $p = 9.5 \times 10^{-34}$), with the optimum at a high confidence and a low IoU threshold. This gain is generic, however: every precision selects nearly the same thresholds, and for INT8 its own calibration ($\tau_c^* = 0.75$, $\tau_i^* = 0.10$) yields $F_1 = 0.785$ versus 0.794 with the transferred FP32 thresholds, a difference of -0.008 [-0.029, +0.013] ($p_{\text{Holm}} = 0.35$). On this workload, quantization-specific threshold calibration therefore offers no measurable benefit over calibrating once on FP32; the calibrated thresholds also cut NMS time from 3.75 to 0.49 ms per image.

E. Decision flips

Aggregate metrics hide how much quantization changes individual outputs (Table III, Fig. 4). FP16 leaves the calibrated detections unchanged, whereas INT8 changes $\Phi = 0.273$ of decisions (227 flips) even at calibrated thresholds, 78% of them false-positive changes; at the default thresholds $\Phi = 0.490$. A deployment that relies on individual detections (e.g. counting or alarms) should therefore validate INT8 at the decision level, not only by mAP.

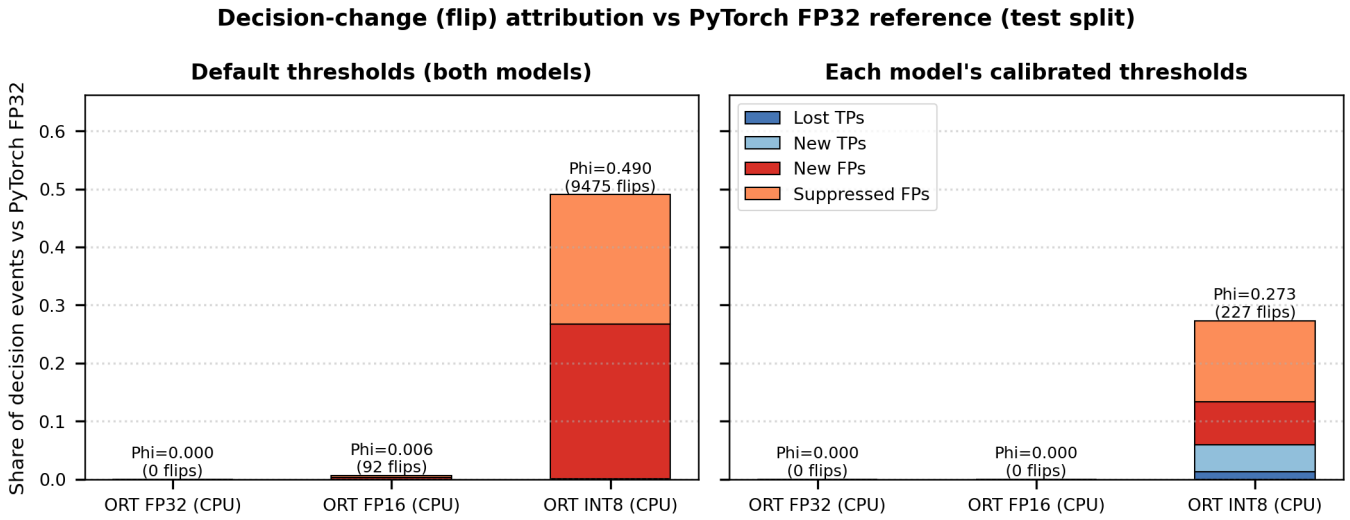


Fig. 4. Decision-flip attribution against PyTorch FP32 at default and calibrated thresholds.

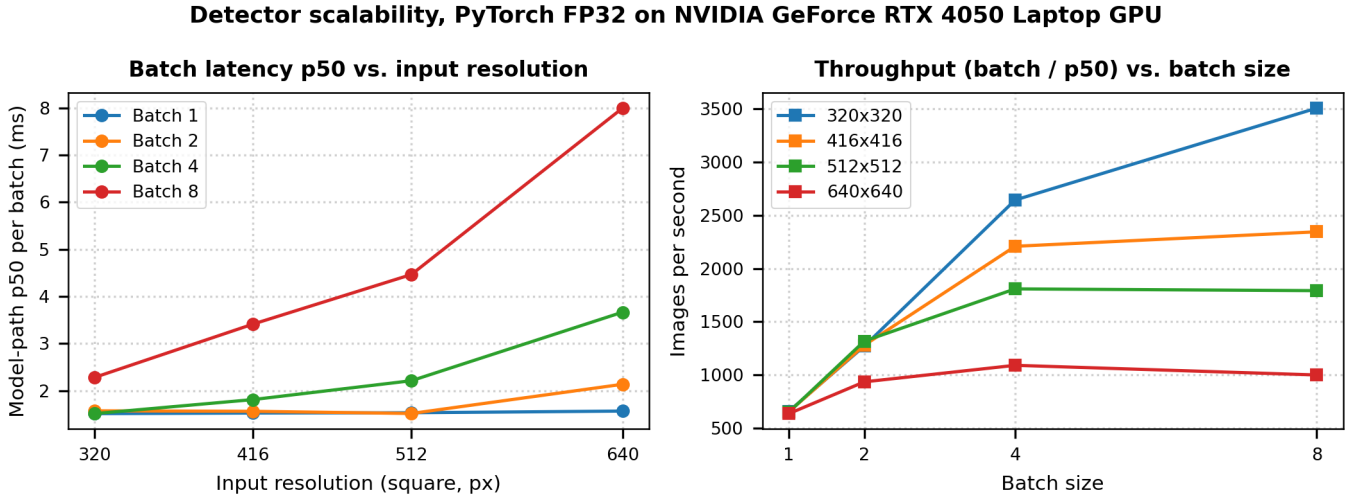


Fig. 5. Detector latency and throughput (batch / p50) across batch size and input resolution, PyTorch FP32 on the GPU.

F. Scalability, memory and stability

At batch 1 the detector's GPU latency is flat across resolutions (1.51 ms at 320 px, 1.57 ms at 640 px), confirming launch-bound execution; batching raises throughput to 3505 images/s at 320 px and 1001 images/s at 640 px with batch 8 (Fig. 5). Peak PyTorch GPU allocations were 35.3 MB (detector) and 44.9 MB (autoencoder) in FP32 (Fig. 6). Every session median stayed within ±5% of its configuration's median for 10 of 12 configurations; the largest deviation was 14.3% (one session of the detector on the GPU in FP32, the configuration that failed the CV criterion; Fig. 7). Tail latency is largest on the GPU at batch 1 (Fig. 8), where host scheduling jitter matters most relative to millisecond-scale kernels.

V. Threats to Validity

Synthetic data and small models. Accuracy numbers describe a controlled synthetic task; they show whether a runtime or precision preserves the trained model's behavior, not how a production detector performs. **Host noise.** Measurements were taken on a laptop under WSL2 with other system activity; the stability verdicts and between-session spread are part of the result rather than something removed by outlier filtering. The bootstrap CIs resample iterations and therefore understate between-session drift. **Hardware scope.** A laptop CPU and GPU are not embedded accelerators; INT8 speedups in particular depend on the CPU's integer instructions and on ONNX Runtime's kernel coverage. **Back-end coverage.** ONNX Runtime CUDA and TensorRT were not available on the host, so GPU comparisons are limited to PyTorch.

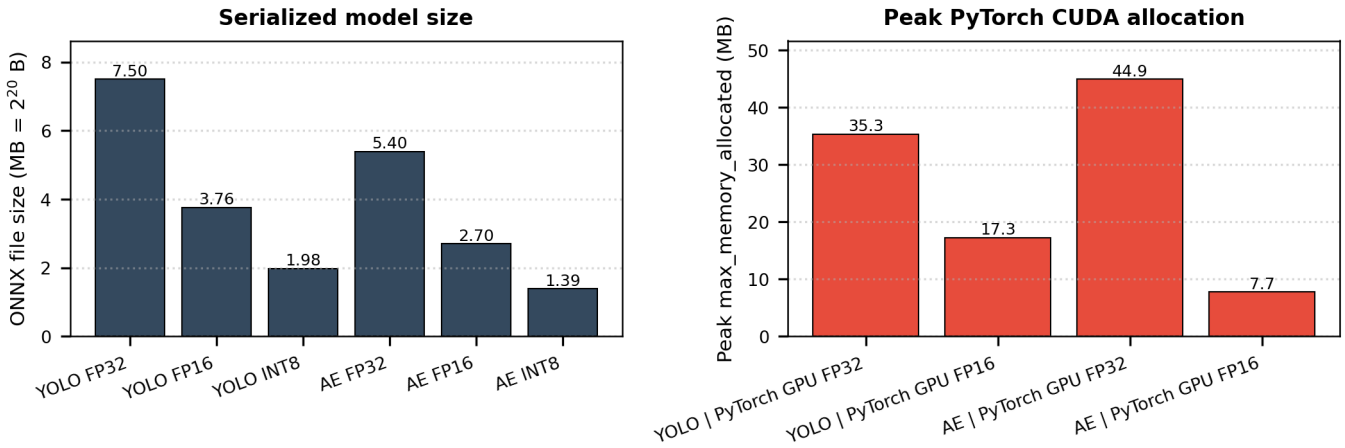


Fig. 6. Serialized model size per precision and peak PyTorch GPU allocation.

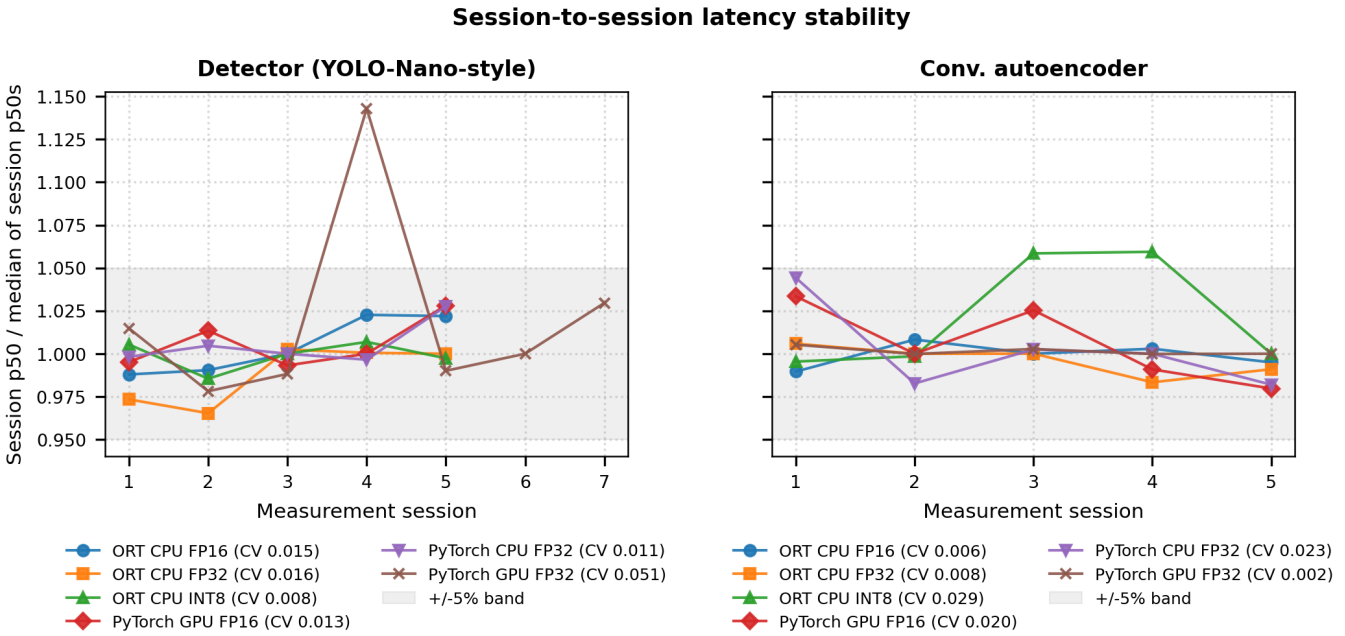


Fig. 7. Per-session median latency normalized by the median across sessions.

VI. Conclusion

Treating latency, parity and task quality as one gated measurement changes several conclusions one would draw from latency alone: the faster CPU runtime depends on the model, static INT8 can be fast yet numerically far from FP32 while its aggregate metric moves in either direction, FP32 on a modern GPU is TF32 unless disabled, and pre- and post-processing dominate end-to-end latency. Calibrating post-processing thresholds matters far more than making them precision-specific: on this workload the FP32-calibrated thresholds transfer to FP16 and INT8 without measurable loss, although INT8 still alters a substantial share of individual detections. Code, configuration, data generators and the recorded session are available at https://github.com/sengarutk/onnx-quant-benchmark; `make pipeline` regenerates every number in this paper.

References

[1] ONNX Runtime developers, "ONNX Runtime," https://onnxruntime.ai, 2021.

[2] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[3] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[4] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.

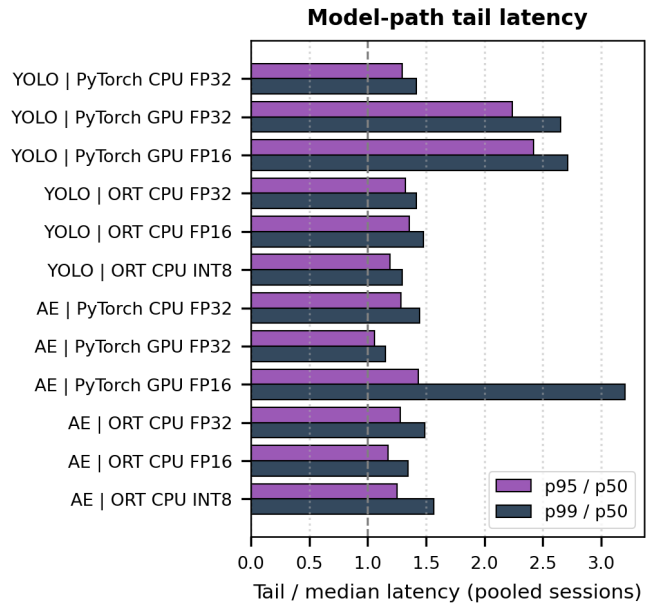


Fig. 8. Tail-to-median latency ratios of the pooled model-path measurements.

[5] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*. Chapman and Hall/CRC, 2022, pp. 291–326.

[6] L. Deng, G. Li, S. Han, L. Shi, and Y. Xie, "Model compression and hardware acceleration for neural networks: A comprehensive survey," *Proceedings of the IEEE*, vol. 108, no. 4, pp. 485–532, 2020.

[7] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[8] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9627–9636.

[9] A. Neubeck and L. V. Gool, "Efficient non-maximum suppression," in *Proceedings of the 18th International Conference on Pattern Recognition (ICPR)*, vol. 3, 2006, pp. 850–855.

[10] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS – improving object detection with one line of code," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 5561–5569.

[11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[12] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 21 002–21 012.

[13] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 658–666.

[14] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

[15] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.

[16] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.

[17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. CRC Press, 1994.

[18] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.